

NaturaLens

Grant Interview Tech Brief

One-pager answers and deeper tech notes for grant / fellowship interviews. Derived from the shipping product, architecture docs, and ML labeling pipeline in the NaturaLens monorepo.

Product	Global wildlife recognition -- photo in, species name + log out
Status	Mobile MVP shipping; landing + waitlist live; labeling + training loop live
Stack	Expo / React Native, Gemini API, Next.js, Cloudflare Workers / D1 / R2
ML path	Cloud-first now -> custom on-device detector (TFLite) when quality bar met
License	MIT
Repo	github.com/abhay-cs/naturalens
Audience	Founders / technical interviewees -- talk tech without drowning in jargon

1. Elevator -- what to say in 60 seconds

NaturaLens is a mobile wildlife recognition app. Point your phone at an animal; we name the species, show confidence and a short species card (habitat, diet, IUCN status), and keep a personal log of finds. Today identification runs in the cloud via Google Gemini. We deliberately ship cloud-first so we can validate the product loop and collect real photos + corrections for a custom on-device model. In parallel we already run an internal labeling product and a polar-bear detector training pipeline aimed at offline, low-latency field use.

Soundbite: 'Cloud to learn fast; on-device to scale privately, cheaply, and offline.'

2. Problem, users, impact (grant framing)

- Problem: Field ID of wildlife is slow, expert-gated, or locked behind heavy apps. Many tools need accounts, maps, or continuous connectivity -- poor fit for quick citizen science and field education.
- Users: Curious outdoors people, students, educators, and eventually researchers who need a lightweight capture -> identify -> log loop on a phone.
- Impact angle: Lower the barrier to species awareness; route interest toward conservation literacy (IUCN status rides with every ID); build a path to privacy-preserving, offline recognition for remote / low-connectivity regions.
- Why tech matters for the grant: Grants fund capability -- not just a UI. Our architecture is explicitly designed to convert usage into labeled training data and then into an owned on-device model (cost, latency, privacy, offline).

3. What exists today vs roadmap

Shipped / working

Mobile app	Expo SDK 54 MVP: capture -> identify -> save -> revisit/delete
Identification	Gemini 3.1 Flash-Lite, structured JSON, ~1.5-2s typical
Local history	AsyncStorage metadata + document-dir photos/thumbs
Landing / waitlist	Next.js on Cloudflare Workers + D1 (naturalens-waitlist)
Labeler (Skim)	Internal tool at skim.naturalens.ca -- upload, box, review, runs
Training loop	SSDLite320 MobileNetV3 -> metrics / preds / TFLite via Colab + R2
Design system	Shared tokens (Outfit + Archivo, B/W chrome) in packages/design

Not started (placeholders in monorepo)

- services/api -- own backend (auth, sightings, species DB, key proxy)
- services/inference -- hosted custom model serving
- tools/data-pipeline -- GBIF / IUCN species DB compilation
- Full multi-species production model + Expo custom-dev-client TFLite runtime

4. System architecture (interview diagram in words)

MVP path (what users touch): Expo app -> resize photo to 1024px JPEG -> Gemini Interactions API with JSON schema -> {isAnimal, label, confidence, description, habitat, diet, conservationStatus} -> optional save to on-device history.

Learning path (what builds the moat): Labeler Worker (Cloudflare) stores images in R2 and labels in D1 -> Create immutable training run (manifest on R2) -> Colab trains SSDLite -> uploads metrics/predictions/TFLite -> UI review of failures -> re-label.

Target production path: App -> our API (key stays server-side) -> cloud model OR on-device TFLite with cloud fallback for low confidence; geotagged finds; species enrichment from GBIF/IUCN; optional regional 'animal packs'.

Honest constraint: Expo Go cannot load a custom TFLite native module -- on-device inference requires a custom development build. That is why Gemini ships first.

5. Tech stack cheat sheet

Mobile	React Native 0.81, React 19, Expo 54, TypeScript, expo-camera
ID model (prod MVP)	gemini-3.1-flash-lite; thinking=minimal; schema-constrained
Persistence	AsyncStorage (versioned key) + expo-file-system document dir
Web / marketing	Next.js 16, Tailwind 4, Framer Motion, Cloudflare Workers
Waitlist DB	Cloudflare D1 (naturalens-waitlist)
Labeler	Cloudflare Worker + D1 (naturalens-labels) + R2 (naturalens-data)
Auth (labeler)	Email+PIN or Cloudflare Access; Colab uses TRAIN_TOKEN
Training	PyTorch torchvision SSDLite320 MobileNetV3-Large; Colab T4 GPU
Export targets	ONNX + TFLite (float16 via onnx2tf when available)
Monorepo	apps / services / models / tools / packages / docs / infra

6. Identification pipeline -- decisions interviewers probe

Q: Why Gemini instead of your own model?

A: Speed to learn. A custom detector needs thousands of labeled images, an eval harness, and a custom Expo build. Gemini lets us ship the product loop now and collect real-world photos + corrections as training fuel.

Q: Why flash-lite specifically?

A: Measured ~1.5s vs ~7-18s for larger flash on the same test images, with the same species answers. Naming an animal does not need a big reasoning model.

Q: Why 1024px uploads?

A: Gemini tokenizes images in fixed tiles -- shrinking to 512px cost detail without saving tokens or latency. 1024px was the sweet spot we measured.

Q: How do you stop non-animals becoming 'species'?

A: Schema includes isAnimal. Without that guard, the model will happily label a chair with confidence 1. Non-animal returns an empty list and the UI says 'No animal here'.

Q: Is confidence calibrated?

A: No -- it is self-reported by the model, clamped to [0,1]. Treat as a UI hint, not a probability. Calibrated scores come with our own trained model + holdout eval.

Q: Why put species facts on the same call?

A: Measured median 1454ms -> 1681ms (~230ms / 16%) for four extra fields. Worth it vs a second round-trip. Older finds without info backfill lazily via a text-only call.

Q: Where does the API key live?

A: In the client today (EXPO_PUBLIC_* is inlined). Restricted in AI Studio, not treated as a secret. Next hard step: proxy via services/api so the key never ships.

7. Data, labeling, and the polar-bear training loop

We built Skim (apps/labeler) as an internal upload / label / train-review app, focused first on a one-class polar bear detector -- a concrete path from photos to a TFLite model without waiting on a full multi-species dataset.

- Storage: images + run artifacts on R2 (naturalens-data); boxes/review/runs on D1.
- Integrity: content-hash (sha256) primary keys; optimistic concurrency (version -> 409); append-only label_history; EXIF orientation normalized at upload.
- Runs: Create training run freezes an immutable runs/<id>/manifest.json on R2.
- Train: Colab notebook pulls snapshot, trains SSDLite, uploads metrics/preds/tflite, callbacks Worker status (TRAIN_TOKEN).
- Cost control: hard internal caps at 80% of R2 free tier (storage / Class A / Class B) so the workflow cannot silently overage -- uploads/runs return HTTP 429 at cap.
- Local split used historically: 44 train / 20 val (models/data/split.json) until a run overrides via R2 manifest.

Dataset size targets (to replace cloud broadly)

Single species (e.g. deer)	Min 1k-2k labeled; better 3k-8k
Two similar species	Min 1.5k-3k per class; better 5k+
10-20 species	Min 1k+ each; better 3k-10k each
Hard negatives	Target 30-50% of samples (no animal / confusers)

8. Cost & latency -- numbers to have ready

Latency

- Our Gemini path: tuned from ~9s down to ~2s (model + size + thinking budget).
- Generic cloud Vision-style on 4G: often 2-9s+ end-to-end -- unreliable for a <1s UX.
- <1s / offline goal: on-device TFLite / Core ML. Cloud stays as fallback for hard cases.

Cloud cost reference (architecture notes -- Vision-like baseline)

- Illustrative Vision combo (~object localization + labels): ~\$3.75 / 1000 photos after free tier. Embeddings for short labels are usually negligible.
- Gemini free tier is usable for MVP/dev; rate limits (429) are a normal user-facing outcome if someone hammers the shutter -- we map that to clear copy.
- Strategic point: per-call cloud cost grows linearly with users; on-device flips that to mostly fixed training + distribution cost.

9. Switch criteria -- cloud -> on-device

Do not claim '95% accuracy' as the only bar. Interview answer:

- Precision / recall per critical class (not just overall accuracy)
- Stable performance on hard conditions (lighting, distance, clutter)
- On-device latency <1s on target phones
- Staged pilot rollout (e.g. 10% -> 50% -> 100%) with cloud fallback
- Admin review of user corrections before training (noise / abuse filter)

10. Competitive / prior art (know Seek)

Seek by iNaturalist validates the endgame pattern: on-device vision + geo ranking, fed by a large observation ecosystem. We are not claiming to out-data them today. Our wedge is a fast personal capture->log product, cloud-first, with an explicit path to owned models and regional packs (species map + compact head + geo priors).

Differentiation talking points: simpler consumer loop; conservation status in-product; owned labeling/training pipeline; MIT-licensed codebase; privacy/offline as design goals.

11. Privacy, safety, ethics

- MVP stores finds only on-device -- no accounts, no sync, no location by design (note: location cannot be backfilled later for old finds).
- Photos sent to Gemini for ID today -- disclose network dependency and third-party processing in interviews / privacy copy.
- On-device future reduces raw photo egress; cloud fallback can be confidence-gated.
- Labeler is internal, authenticated; training data should prefer consented / owned field photos and reviewed labels.
- Conservation status is schema-enum constrained; unknown -> Data Deficient rather than inventing a confident IUCN category.

12. Risks & mitigations (grant panels love this)

API key in client	Restrict key; next: proxy via services/api
No offline field use yet	Roadmap to TFLite; cloud is interim
Uncalibrated confidence	Clamp + UI as hint; own model + eval later
Label noise / abuse	Admin review queue; append-only history; versioning
Cloud cost at scale	On-device migration + regional packs
Expo Go TFLite limit	Custom dev client when model ready
Infra free-tier blowup	R2 hard caps at 80%; D1-only run lists
No automated tests yet	Acknowledged gap; CI typechecks mobile on main

13. Roadmap talking track

- Now: tighten MVP, proxy Gemini, geotag + map, export history
- Next: species DB enrichment (GBIF/IUCN); grow labeled sets beyond polar bear
- Later: multi-species on-device model; animal packs (region priors); cloud fallback

- Always: corrections -> reviewed labels -> retrain -> evaluate -> gradual rollout

14. Rapid-fire Q&A bank

Q: What is innovative here?

A: Not 'another classifier' alone -- the closed loop: consumer ID product that funds a labeled real-world dataset, plus an operational labeling/training stack already running toward offline models.

Q: Why a monorepo?

A: Small team, atomic PRs across app + ML + docs, one clone for onboarding (ADR 001). Boundaries: apps / services / models / tools.

Q: Open source?

A: MIT. Easy for collaborators and academic partners; still compatible with hosted services and future sustainability models.

Q: Who is on the critical path technically?

A: Mobile product loop (Expo), identification quality/UX, labeling throughput, and the train->eval->export path to TFLite. Backend API is the next production hardening step.

Q: What would grant funding unlock?

A: Concrete uses: expand labeled datasets across species/regions; GPU/time for training & eval; build services/api + inference; ship custom on-device builds; GBIF/IUCN enrichment; field pilots with educators/partners.

Q: How do you measure success?

A: Product: successful IDs, save rate, retention of history use. ML: per-class P/R, hard-case stability, on-device latency, pilot error rates vs cloud. Ops: labeling throughput and reviewed-correction volume.

Q: Why polar bear first for custom ML?

A: One-class detector is a tractable vertical slice -- proves upload->label->train->TFLite end-to-end before multi-species complexity. Field-photo focused; transferable pipeline.

Q: What about maps / location?

A: Intentionally out of MVP. Planned next. Important honesty: finds saved before location ships will never get backfilled coordinates.

Q: Live video detection?

A: Out of MVP. One photo on button press -- the frozen still is what was analyzed. Live detection returns with on-device models.

Q: How is this not just wrapping Gemini?

A: Gemini is the current inference backend. Product UX, persistence, species card design, error handling, and -- critically -- the labeling + training pipeline that makes Gemini replaceable are the durable work.

15. Component status snapshot

apps/mobile	MVP -- identify + local history
apps/web	Built -- landing + D1 waitlist on Workers
apps/labeler	Built -- Skim labeling / runs / Colab loop
services/api	Not started (placeholder)

services/inference	Not started (placeholder)
models/training	SSDLite polar-bear pipeline + Colab
models/evaluation	Placeholder README -- metrics emitted by training
tools/data-pipeline	Not started (GBIF/IUCN planned)
infra/	Not started (Docker/IaC planned)

16. Glossary (say it cleanly)

Expo Go	Scan-QR RN runtime; cannot add custom native ML modules
TFLite	TensorFlow Lite -- mobile on-device inference format
SSDLite	Lightweight single-shot detector; our current train target
D1 / R2	Cloudflare SQL DB / object storage used by web + labeler
GBIF	Global Biodiversity Information Facility -- occurrence/taxonomy data
IUCN	Red List conservation categories we surface in-product
mAP	mean Average Precision -- detection quality metric on val set
Animal pack	Future region pack: labels + compact model/index + geo priors

17. Closing line

NaturaLens ships a real capture->identify->log product today, while building the data and training machinery to own recognition tomorrow -- private, offline-capable, and cost-stable at scale. Grant support accelerates the hard part: data, models, and the production backend that turns a strong MVP into durable infrastructure.

Sources in-repo: README.md, docs/DESIGN.md, docs/Architecture_Notes.md, docs/adr/001-monorepo.md, apps/*/README.md, models/training/README.md. Regenerate this PDF: `python3 docs/grant-interview/build_pdf.py`